

Rajalakshmi Engineering College

Name: Harsan J
Email: 240701179@rajalakshmi.edu.in
Roll no: 240701179
Phone: 8925466195
Branch: REC
Department: CSE - Section 10
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 10_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : COD

1. Problem Statement

The city library maintains a record of books available for lending. Each book is uniquely identified by its ISBN number, along with its title and author. The librarian wants to efficiently store and manage these records, ensuring books can be listed in the order they were added.

Your task is to implement a Library Management System using HashSet where:

The librarian adds books with ISBN, title, and author. The librarian can remove books by providing an ISBN. Finally, the librarian displays the available books in the order they were added.

Implement a class Library that will handle these operations. The main function should manage user input and interact with the Library class accordingly.

Input Format

The first line contains an integer n – the number of books to be added.

The next n lines contain three values: ISBN (integer), Title (string without spaces), and Author (string without spaces).

1. An integer `employee_id`
2. A string `title`
3. A string `author name`

The next line contains an integer m – the number of books to be removed.

The next m lines follow, each contains an ISBN number to remove.

Output Format

The output prints a list of books available in the library after performing all operations in the format:

"ISBN: <isbn>, Title: <title>, Author: <author>"

If no books remain, print: "No books available"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

1234 JavaCompleteGuide JohnDoe

5678 PythonBasics JaneDoe

9012 DataStructures AliceSmith

1

5679

Output: ISBN: 1234, Title: JavaCompleteGuide, Author: JohnDoe

ISBN: 9012, Title: DataStructures, Author: AliceSmith

ISBN: 5678, Title: PythonBasics, Author: JaneDoe

Answer

```
import java.util.*;
```

```

class book{
    int a;
    String b,c;
    book(int a,String b,String c)
    {
        this.a=a;
        this.b=b;
        this.c=c;
    }
    public String toString()
    {
        return "ISBN: "+a+", Title: "+b+", Author: "+c;
    }
    public int hashCode()
    {
        return Objects.hash(a,b,c);
    }
    public boolean equals(Object obj)
    {
        if(this==obj) return true;
        if(!(obj instanceof book)) return false;
        book l=(book)obj;
        return a==l.a && b.equals(l.b) && c.equals(l.c);
    }
}

```

```

class Library
{
    HashSet <book> set = new HashSet<>();
    public void addBook(int a,String b,String c)
    {
        set.add(new book(a,b,c));
    }
    public void removeBook(int n)
    {
        book tor=null;
        for(book l : set)
        {
            if(l.a==n)
            {
                tor=l;
                break;
            }
        }
    }
}

```

```
}
    if(tor!=null)
    {
        set.remove(tor);
    }

}

public void displayBooks()
{

    if(set.isEmpty())
    {
        System.out.print("No books available");
    }
    else
    {
        for(book t : set)
        {
            System.out.println(t);
        }
    }
}
}
```

```
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Library library = new Library();
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            int isbn = sc.nextInt();
            String title = sc.next();
            String author = sc.next();
            library.addBook(isbn, title, author);
        }
        int m = sc.nextInt();
```

```
for (int i = 0; i < m; i++) {  
    int isbn = sc.nextInt();  
    library.removeBook(isbn);  
}  
library.displayBooks();  
sc.close();  
}  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Aryan is developing a voting system for a college election. Each vote is recorded as an entry in an array, where every student's vote is represented by a candidate's ID. Since it's a majority-rule election, the winner is the candidate who receives more than $n/2$ votes, where n is the total number of votes cast.

To quickly determine the winner, Aryan decides to use a HashMap to count the occurrences of each vote and identify the candidate who has received more than half of the total votes.

Example

Input

7

2 2 1 2 2 3

Output

2

Explanation

The votes are: 2, 2, 1, 2, 2, 3, 2

Count of each candidate:

2 appears 5 times 1 appears once 3 appears once

The majority element is the one that appears more than $N/2$ times. Since

$7/2 = 3.5$, a number must appear at least 4 times to be the majority.

The number 2 appears 5 times, which is greater than 3.5, so the output is 2.

Input Format

The first line contains an integer N representing the number of votes cast.

The second line contains N space-separated integers representing the votes, where each integer corresponds to a candidate.

Output Format

The output prints an integer representing the majority element (the candidate who received more than $N/2$ votes).

If no such candidate exists, print -1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

2 2 1 2 2 2 3

Output: 2

Answer

```
import java.util.HashMap;
```

```
import java.util.Scanner;
```

```
import java.util.*;
```

```
class MajorityElementFinder {  
    public static int findMajorityElement(int[] arr) {  
        HashMap<Integer, Integer> map = new HashMap<>();  
        int n = arr.length;  
        for (int num : arr) {  
            map.put(num, map.getDefault(num, 0) + 1);  
        }  
        for (Map.Entry<Integer, Integer> entry : map.entrySet()) {  
            if (entry.getValue() > n / 2) {
```

```
        return entry.getKey();
    }
}

return -1;
}
}
```

```
class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int N = scanner.nextInt();
        int[] arr = new int[N];

        for (int i = 0; i < N; i++) {
            arr[i] = scanner.nextInt();
        }

        int result = MajorityElementFinder.findMajorityElement(arr);
        System.out.println(result);

        scanner.close();
    }
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Tony is an e-learning platform administrator, he oversees the user ratings for various online courses offered in the platform.

To enhance user experience, you should assist him in utilizing a HashMap to store course ratings given by learners. Regularly, he analyzes this data to identify the highest and lowest-rated courses, enabling targeted improvements and ensuring the quality of the educational content. This process assists in maintaining a competitive and engaging online learning environment for the users.

Input Format

The input consists of a string representing the course name followed by a double value representing the course's rating, in separate lines.

The input is terminated by entering "done".

Output Format

The first line of output prints the string "Highest Rated Course: " followed by the highest-rated course.

The second line prints the string "Lowest Rated Course: " followed by the lowest-rated courses.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: DSA

4.0

OOPS

4.2

C

3.2

done

Output: Highest Rated Course: OOPS

Lowest Rated Course: C

Answer

```
import java.util.HashMap;
```

```
import java.util.Map;
```

```
import java.util.Scanner;
```

```
// You are using Java
```

```
class CourseAnalyzer
```

```
{
```

```
    public Map<String,String>
```

```
    identifyHighestAndLowestRatedCourses(Map<String,Double> course)
```

```
{
```

```
        String hc="";
```



```

String lc="";
double hr=Double.MIN_VALUE;
double lr=Double.MAX_VALUE;
for(Map.Entry<String,Double>entry:course.entrySet())
{
    String c=entry.getKey();
    double r=entry.getValue();
    if(r>hr)
    {
        hr=r;
        hc=c;
    }
    if(r<lr)
    {
        lr=r;
        lc=c;
    }
}

Map<String,String>result=new HashMap<>();
result.put("highest",hc);
result.put("lowest",lc);
return result;
}

}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Map<String, Double> courseRatings = new HashMap<>();

        while (true) {
            String courseName = scanner.nextLine();
            if (courseName.equalsIgnoreCase("done")) {
                break;
            }
            double rating = Double.parseDouble(scanner.nextLine().trim());
            courseRatings.put(courseName, rating);
        }

        CourseAnalyzer analyzer = new CourseAnalyzer();
        Map<String, String> result =
        analyzer.identifyHighestAndLowestRatedCourses(courseRatings);
    }
}

```

```
System.out.printf("Highest Rated Course: %s\n", result.get("highest"));
System.out.printf("Lowest Rated Course: %s", result.get("lowest"));

    scanner.close();
}
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Bob wants to develop a score-tracking application for a gaming tournament. Each player's score is stored in a HashMap with the player's name as the key and the score as the value.

Write a program to assist Bob that takes user input to enter player scores, calculates the maximum score from the HashMap, and prints the player with the highest score.

Input Format

The input consists of strings representing player details in the format "playerName:score".

The input is terminated by entering "done".

Output Format

The output displays a string, representing the player's name who scored the maximum.

If the value is not numeric, print "Invalid input".

If any special characters other than ':' are given, print "Invalid format".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Alice:15

Bob:56

done

Output: Bob

Answer

```
import java.util.*;
```

```
// You are using Java
```

```
class ScoreTracker
```

```
{
```

```
    int m=0;
```

```
    HashMap<String,Integer> scoreMap=new HashMap<>();
```

```
    public boolean processInput(String s)
```

```
    {
```

```
        if(!s.contains(":") && m==0)
```

```
        {
```

```
            System.out.print("Invalid format");
```

```
            m=1;
```

```
            return false;
```

```
        }
```

```
        else
```

```
        {
```

```
            String[] p=s.split(":");
```

```
            if(!isInteger(p[1]))
```

```
            {
```

```
                m=1;
```

```
                System.out.print("Invalid input");
```

```
                return false;
```

```
            }
```

```
            else
```

```
            {
```

```
                scoreMap.put(p[0],Integer.parseInt(p[1]));
```

```
                return true;
```

```
            }
```

```
        }
```

```
    }
```

```
    public boolean isInteger(String s)
```

```
    {
```

```
        try
```

```

    {
        Integer.parseInt(s);
        return true;
    }
    catch(NumberFormatException e)
    {
        return false;
    }
}

public String findTopPlayer()
{
    Map.Entry<String,Integer>
    top=Collections.max(scoreMap.entrySet(),Map.Entry.comparingByValue());
    return top.getKey();
}
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ScoreTracker tracker = new ScoreTracker();
        boolean validInput = true;

        while (true) {
            String input = scanner.nextLine();

            if (input.toLowerCase().equals("done")) {
                break;
            }

            if (!tracker.processInput(input)) {
                validInput = false;
                break;
            }
        }

        if (validInput && !tracker.scoreMap.isEmpty()) {
            System.out.println(tracker.findTopPlayer());
        }

        scanner.close();
    }
}

```

Status : Correct

Marks : 10/10